

MANGESH SOLUTION

REACT.JS DEVELOPMENT

Complete Student Notes • Concepts • Examples • Exam & Interview Revision

Prepared for Mangesh Solution

A practical learning reference for students and beginners learning modern React.js development.

Core idea: React helps you build user interfaces from reusable components. The most important flow is Component → Props/State → Event → Render → Update.

Contents

No.	Topic
1	React Fundamentals
2	Setup & Project Structure
3	JSX
4	Components
5	Props
6	State & useState
7	Events & Forms
8	Conditional Rendering & Lists
9	Hooks
10	useEffect & API Calls
11	Context API
12	Routing
13	Component Communication
14	Forms & Validation
15	API Integration
16	Performance
17	Best Practices
18	Common Mistakes
19	Interview Questions
20	Quick Revision

1. React Fundamentals

React.js is a JavaScript library for building user interfaces, especially modern single-page applications. It encourages a component-based approach, where a large UI is divided into small reusable pieces.

- Component-based: UI is divided into reusable components.
- Declarative: You describe the UI for a particular state.
- Virtual DOM: React efficiently updates the browser DOM.
- One-way data flow: Data commonly moves parent → child through props.
- Reusable: The same component can be used in multiple places.

2. Setup & Project Structure

A common modern way to start a React project is Vite.

Create a project:

```
npm create vite@latest my-react-app
cd my-react-app
npm install
npm run dev
```

File/Folder	Purpose
src/	Application source code
src/main.jsx	Entry point that mounts React
src/App.jsx	Main application component
public/	Static public assets
package.json	Dependencies and scripts

3. JSX

JSX is a syntax extension that lets you write HTML-like markup inside JavaScript. JSX is transformed into JavaScript before it runs in the browser.

- Use className instead of HTML class.
- JavaScript expressions are written inside { }.
- Tags must be closed.
- A component can return a Fragment: <>...</>.

```
function Welcome() {
  const name = "Mangesh";
  return <h1>Hello, {name}!</h1>;
}
```

4. Components

A component is an independent, reusable piece of UI. Modern React commonly uses function components.

- Keep components focused and reusable.
- Component names start with a capital letter.
- Split a large UI into logical components.

5. Props

Props (properties) are inputs passed from a parent component to a child. A child should treat props as read-only.

- Props = incoming data.
- Props can contain strings, numbers, objects, arrays and functions.
- A callback prop can let a child communicate an event to its parent.

```
function UserCard({ name, course }) {  
  return <p>{name} is learning {course}</p>;  
}  
  
function App() {  
  return <UserCard name="Mangesh" course="React.js" />;  
}
```

Easy difference: Props are passed into a component; state is managed by the component.

6. State & useState

State is data managed by a component that can change over time. Updating state causes React to render the component again.

- Do not mutate state directly.
- Use the setter returned by useState.
- When the next state depends on the previous state, prefer the functional update form.

```
import { useState } from "react";  
  
function Counter() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <button onClick={() => setCount(count + 1)}>  
      Count: {count}  
    </button>  
  );  
}
```

7. Events & Forms

React uses event handlers such as onClick, onChange and onSubmit. Controlled inputs keep their values in React state.

- Call e.preventDefault() when appropriate for form submission.
- Use state for controlled input values.
- Validate user input and show useful messages.

```
function Login() {  
  const [email, setEmail] = useState("");  
  
  function handleSubmit(e) {  
    e.preventDefault();  
    console.log(email);  
  }  
  
  return (  
    <form onSubmit={handleSubmit}>  
      <input value={email}  
        onChange={(e) => setEmail(e.target.value)} />  
      <button type="submit">Login</button>  
    </form>  
  );  
}
```

```
);  
}
```

8. Conditional Rendering & Lists

Use JavaScript conditions to decide what UI should appear. Use `map()` to render arrays and provide each item with a stable key.

- Use the ternary operator for two alternatives.
- Use `&&` for simple conditional display.
- Prefer stable unique IDs as list keys when list order can change.

```
{isLoggedIn ? <Dashboard /> : <Login />}  
  
{error && <p>{error}</p>}  
  
{users.map(user => (  
  <li key={user.id}>{user.name}</li>  
))}
```

9. Hooks

Hooks let function components use React features such as state, effects, context and refs.

- `useState`: state.
- `useEffect`: external synchronization / side effects.
- `useContext`: context values.
- `useRef`: DOM references or mutable values.
- `useMemo/useCallback`: performance-related memoization when useful.

Hook	Main Use
<code>useState</code>	Component state
<code>useEffect</code>	External synchronization / side effects
<code>useContext</code>	Read shared context
<code>useRef</code>	DOM reference / mutable value
<code>useMemo</code>	Memoize an expensive calculation
<code>useCallback</code>	Memoize a function reference

Rules of Hooks: Call hooks at the top level of a React function component or custom hook. Do not call them inside loops, conditions, or nested ordinary functions.

10. `useEffect` & API Calls

`useEffect` is used to synchronize a component with an external system, such as a network request, subscription, timer, or browser API. It should not be used simply because code needs to run after rendering.

- Keep loading, success, empty and error states in mind.
- Declare dependencies correctly.
- Clean up subscriptions/timers when needed.

```
useEffect(() => {
  async function loadUsers() {
    const response = await fetch("/api/users");
    if (!response.ok) throw new Error("Request failed");
    const data = await response.json();
    setUsers(data);
  }
  loadUsers();
}, []);
```

11. Context API

Context lets components access shared values without passing props through every intermediate component. It is useful for values such as theme, language or authenticated-user information.

- Do not put every state variable into Context.
- Keep context values focused on genuinely shared data.

```
const ThemeContext = createContext("light");

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Dashboard />
    </ThemeContext.Provider>
  );
}

function Dashboard() {
  const theme = useContext(ThemeContext);
  return <div>Theme: {theme}</div>;
}
```

12. Routing

Client-side routing lets a React application show different views based on the URL without a full browser page reload for normal navigation. React Router is a common choice.

- Routes map URL paths to UI.
- Use navigation links for SPA navigation.
- Protect authenticated routes in the application architecture when needed.

```
import { BrowserRouter, Routes, Route } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```

13. Component Communication

Parent-to-child communication normally uses props. Child-to-parent communication commonly uses a callback function passed from the parent.

- Parent → Child: props.
- Child → Parent: callback prop.

- Shared across many components: Context or a state-management solution.

```
function Parent() {
  const handleMessage = (message) => console.log(message);
  return <Child onSend={handleMessage} />;
}

function Child({ onSend }) {
  return <button onClick={() => onSend("Hello!")}>Send</button>;
}
```

14. Forms & Validation

Forms usually keep important field values in state. Validate on the client for a good user experience and validate again on the server for security and correctness.

- Show field-specific errors.
- Prevent duplicate submissions where appropriate.
- Never treat client-side validation as a security boundary.

```
const [form, setForm] = useState({ name: "", email: "" });

function handleChange(e) {
  setForm({
    ...form,
    [e.target.name]: e.target.value
  });
}
```

15. API Integration

A typical flow is: user action → API request → backend → response → state update → UI re-render.

- Check response status.
- Handle network/server errors.
- Show loading and empty states.
- Never put private API secrets in browser code.

```
async function loadProducts() {
  const response = await fetch("/api/products");
  if (!response.ok) throw new Error("Failed to load products");
  return response.json();
}
```

16. Performance

React applications should be optimized based on real performance needs rather than adding memoization everywhere.

- Use React.memo when it meaningfully prevents re-renders.
- useMemo can cache expensive calculations.
- useCallback can stabilize function references when that matters.
- Lazy-load large features when appropriate.
- Use stable list keys.

17. Best Practices

As projects grow, a clear structure makes code easier to maintain.

- Use components/ for reusable UI.
- Use pages/ for page-level screens.
- Use services/ for API functions in larger projects.
- Use hooks/ for reusable stateful logic.
- Use meaningful names and avoid giant components.
- Keep secrets on the server, not in client-side source code.

18. Common Mistakes

These mistakes are common when learning React.

- Mutating state directly → use the setter.
- Missing list keys → use stable IDs.
- Calling hooks inside conditions → call hooks at the top level.
- Ignoring API errors → handle loading/error/success.
- Using useEffect for every calculation → derive values during render when possible.
- Overusing Context → keep shared state purposeful.

19. Important Interview & Exam Questions

Q. What is React.js?

Answer: A JavaScript library for building user interfaces with reusable components.

Q. What is JSX?

Answer: A syntax extension that allows HTML-like markup inside JavaScript.

Q. What is a component?

Answer: A reusable unit of UI and logic.

Q. What are props?

Answer: Read-only inputs passed from a parent component to a child.

Q. What is state?

Answer: Data managed by a component that can change and cause a re-render.

Q. What is useState?

Answer: A hook used to add state to a function component.

Q. What is useEffect?

Answer: A hook used to synchronize a component with external systems and perform side effects.

Q. Why are keys used in lists?

Answer: They provide stable identity so React can track list items across updates.

Q. What is Context?

Answer: A mechanism for making shared values available to components without prop drilling.

Q. What is SPA?

Answer: A single-page application that changes displayed views without full page reloads for normal client-side navigation.

20. Quick Revision Sheet

Concept	Remember
React	UI library based on reusable components
JSX	HTML-like syntax inside JavaScript
Props	Parent → child data
State	Component-managed changing data
useState	Creates and updates state
useEffect	External synchronization / side effects
Context	Shared values across component tree
Router	URL-based client-side views
map()	Render arrays/lists
key	Stable identity for list items

Concept	Remember
API	Frontend ↔ backend communication

Golden React flow: User interaction → event handler → state update → React render → updated UI.

Mangesh Solution — Practical Learning: Use these concepts to build real projects such as dashboards, admin panels, student management systems and full-stack web applications.

Practice project: Build a Student Management app with Add Student, Edit/Delete, Search, validation, API integration, loading/error states and routing. It covers most React fundamentals.